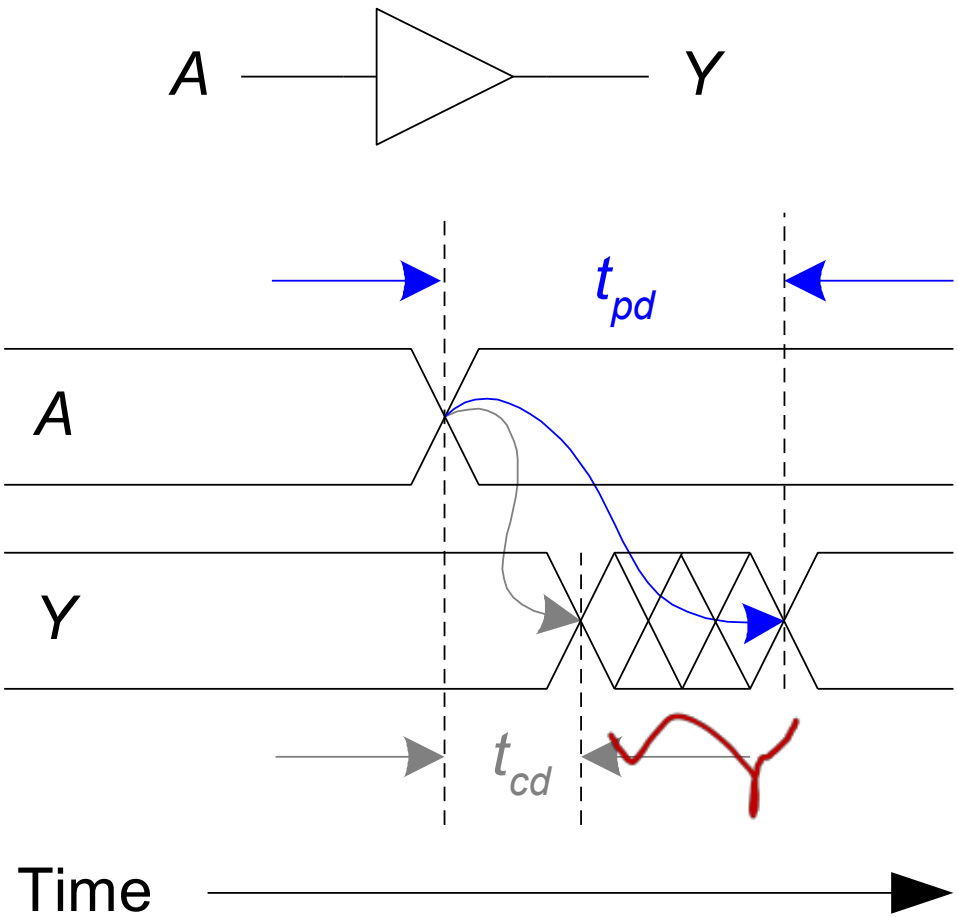
ECE 2011
Introduction to Computer Engineering
Spring 2026

Lecture 10
02-12-2026

Linghao Song
Yale ECE

Propagation & Contamination Delay

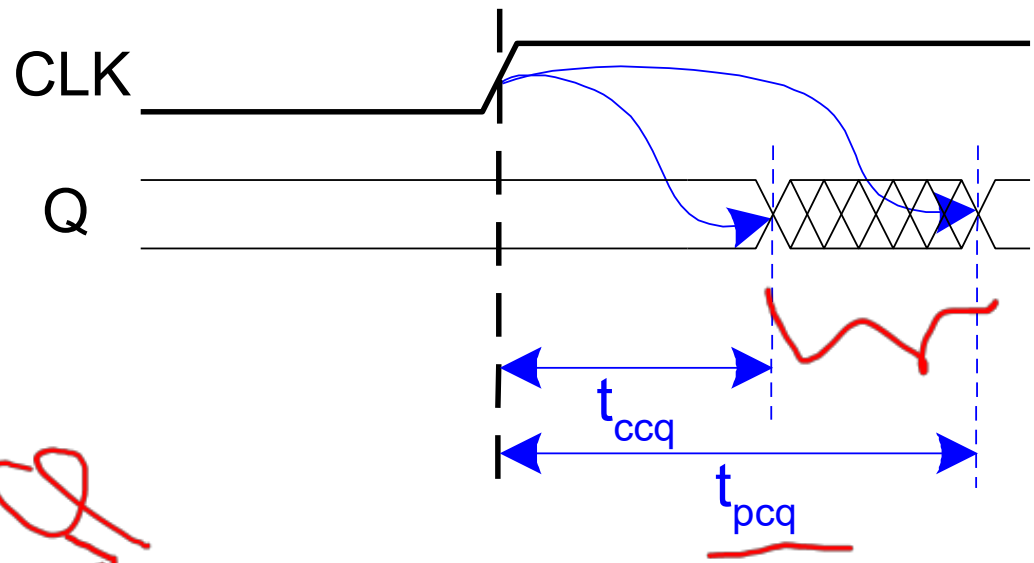
- **Propagation delay:** t_{pd} = max delay from input to output
- **Contamination delay:** t_{cd} = min delay from input to output

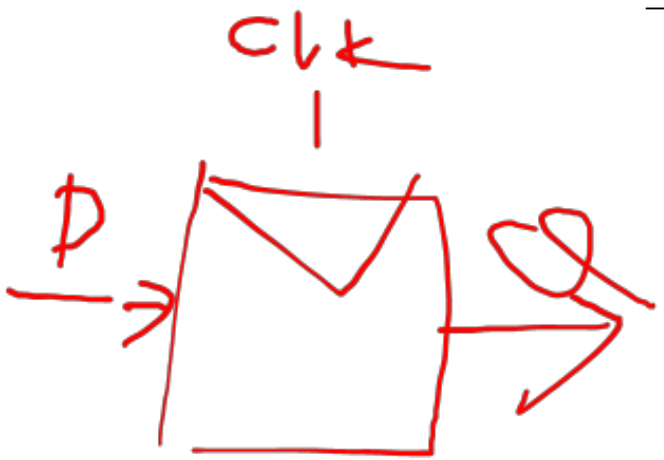


© *Digital Design and Computer Architecture*, 2nd Edition, 2012

Output Timing Constraints

- **Propagation delay:** t_{pcq} = time after clock edge that the output *Q* is guaranteed to be stable (i.e., to stop changing)
- **Contamination delay:** t_{ccq} = time after clock edge that *Q* might be unstable (i.e., start changing)



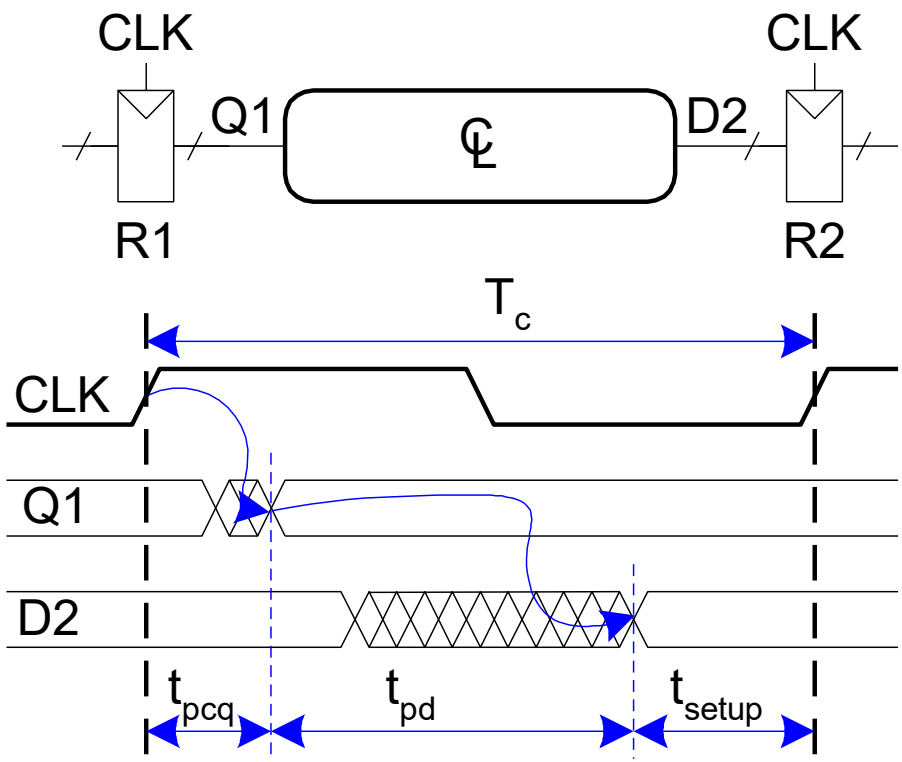


© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Setup Time Constraint

- Depends on the **maximum** delay from register R1 through the combinational logic to R2
- The input to register R2 must be stable at least t_{setup} before clock edge



$$T_c \geq t_{pcq} + t_{pd} + t_{setup}$$

$$t_{pd} \leq T_c - (t_{pcq} + t_{setup})$$

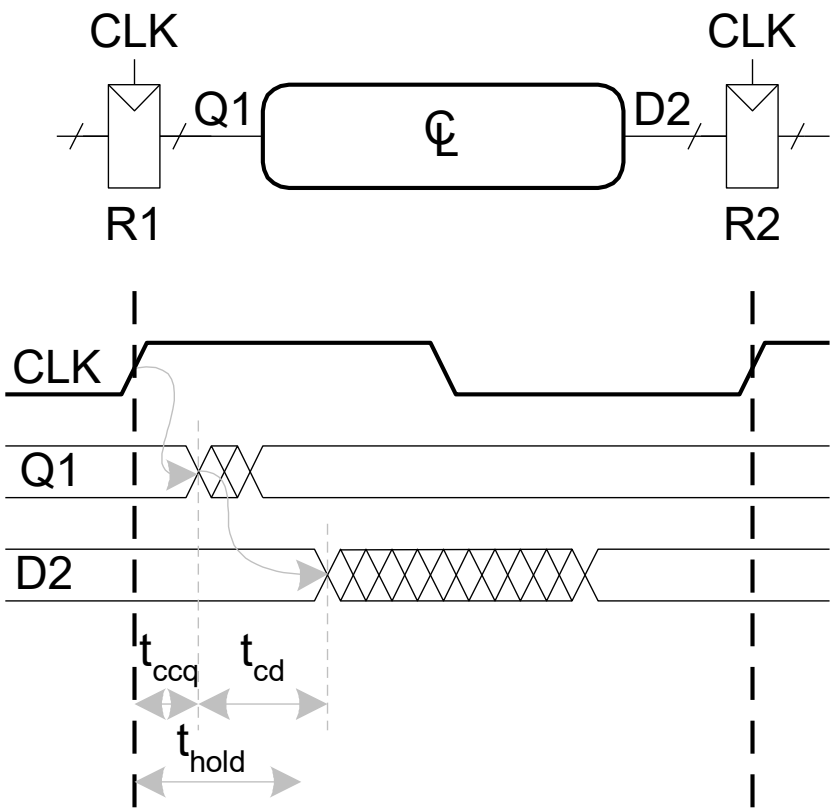
$(t_{pcq} + t_{setup})$: *sequencing overhead*

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Hold Time Constraint

- Depends on the **minimum** delay from register R1 through the combinational logic to R2
- The input to register R2 must be stable for at least t_{hold} after the clock edge



$$t_{hold} < t_{ccq} + t_{cd}$$

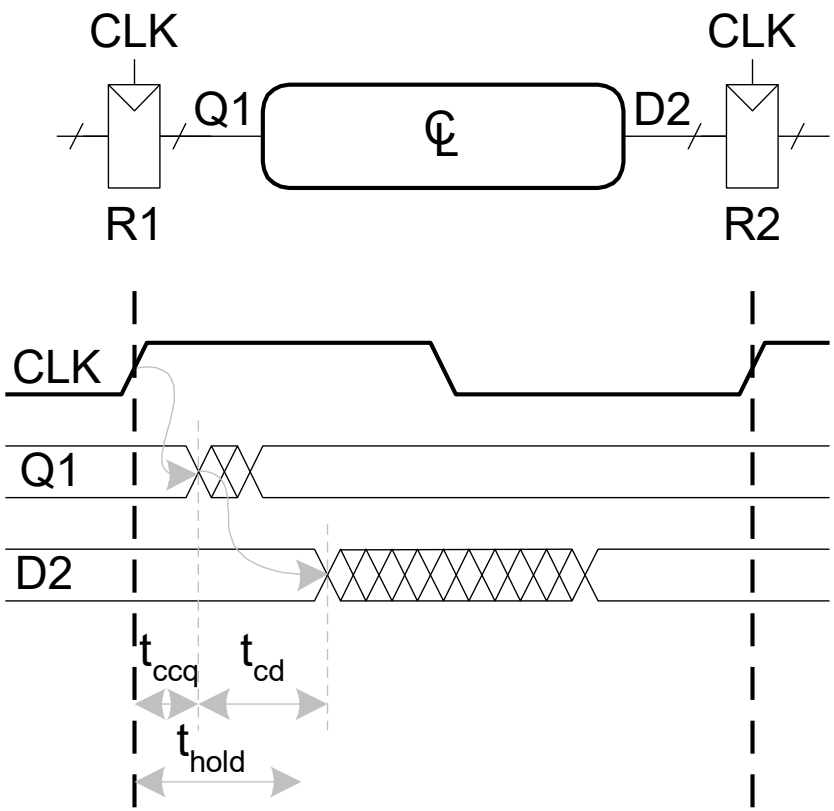
$$t_{cd} >$$

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Hold Time Constraint

- Depends on the **minimum** delay from register R1 through the combinational logic to R2
- The input to register R2 must be stable for at least t_{hold} after the clock edge



$$t_{hold} < t_{ccq} + t_{cd}$$

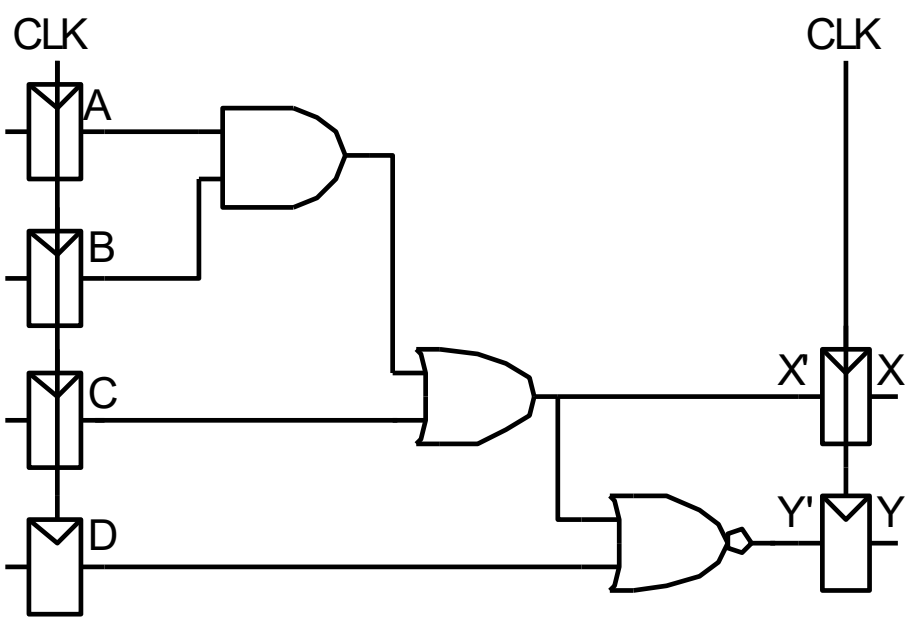
$$t_{cd} > t_{hold} - t_{ccq}$$

Usually, registers are designed that $t_{hold} = 0$, so we don't need to worry (unless there is a explicit $t_{hold} != 0$).

© Digital Design and Computer Architecture, 2nd Edition, 2012

ELSEVIER

Timing Analysis



Timing Characteristics

t_{ccq} = 30 ps

t_{pcq} = 50 ps

t_{setup} = 60 ps

t_{hold} = 70 ps

per gate

t_{pd} = 35 ps

t_{cd} = 25 ps

t_{pd} =

t_{cd} =

Setup time constraint:

$T_c \geq$

f_c =

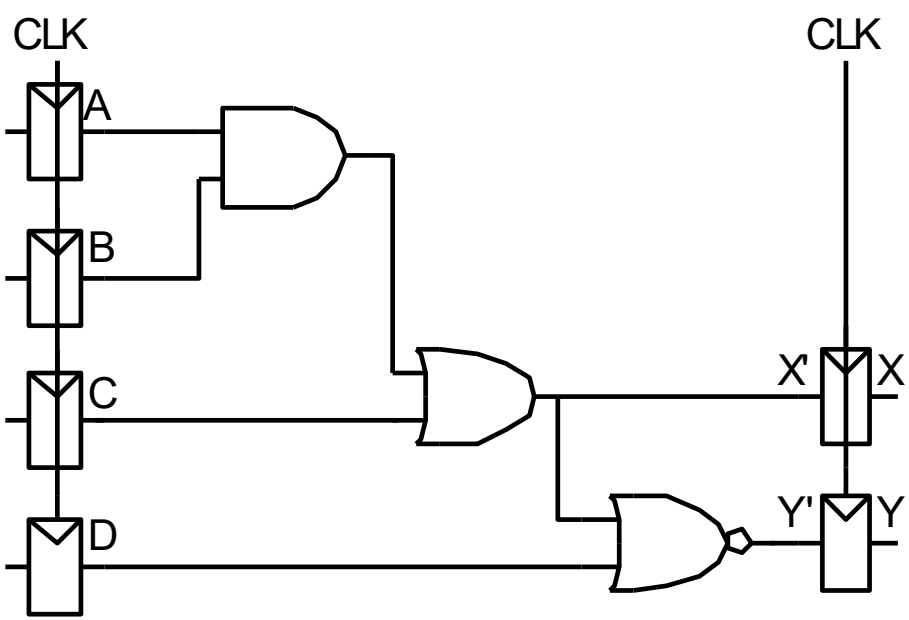
Hold time constraint:

$t_{ccq} + t_{cd} > t_{hold}$?

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Timing Analysis



Timing Characteristics

t_{ccq} = 30 ps

t_{pcq} = 50 ps

t_{setup} = 60 ps

t_{hold} = 70 ps

per gate

t_{pd} = 35 ps

t_{cd} = 25 ps

t_{pd} = 3 x 35 ps = 105 ps

t_{cd} = 25 ps

Setup time constraint:

$T_c \geq (50 + 105 + 60)$ ps = 215 ps

$f_c = 1/T_c$ = 4.65 GHz

Hold time constraint:

$t_{ccq} + t_{cd} > t_{hold}$?

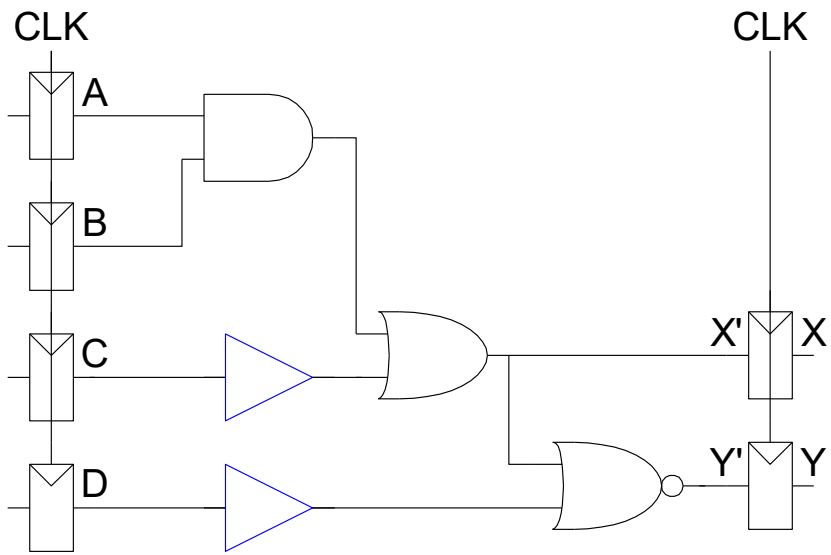
(30 + 25) ps > 70 ps ? **No!**

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Timing Analysis

Add buffers to the short paths:



Timing Characteristics

t_{ccq} = 30 ps

t_{pcq} = 50 ps

t_{setup} = 60 ps

t_{hold} = 70 ps

per gate

t_{pd} = 35 ps

t_{cd} = 25 ps

t_{pd} =

t_{cd} =

Setup time constraint:

$T_c \geq$

f_c =

Hold time constraint:

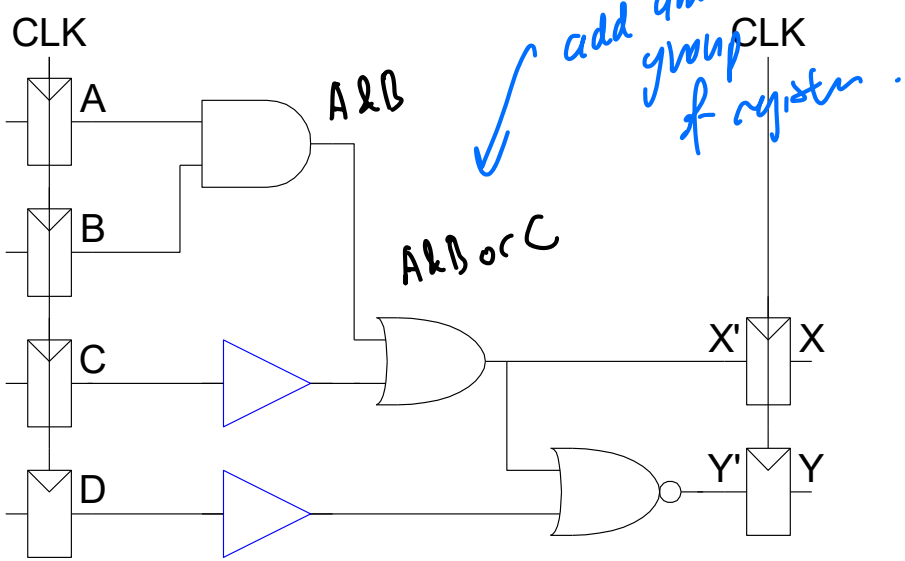
$t_{ccq} + t_{cd} > t_{hold}$?

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Timing Analysis

Add buffers to the short paths:



$T_c \geq t_{pcq} + t_{pd} + t_{setup}$

t_{pd} = 3 x 35 ps = 105 ps

t_{cd} = 2 x 25 ps = 50 ps

Setup time constraint:

$T_c \geq$ (50 + 105 + 60) ps = 215 ps

$f_c = 1/T_c$ = 4.65 GHz

if I want f_c = 5GHz ⇒ T_c = 200 ps.

Timing Characteristics

t_{ccq} = 30 ps

t_{pcq} = 50 ps

t_{setup} = 60 ps

t_{hold} = 70 ps

per gate

t_{pd} = 35 ps

t_{cd} = 25 ps

Hold time constraint:

$t_{ccq} + t_{cd} > t_{hold}$?

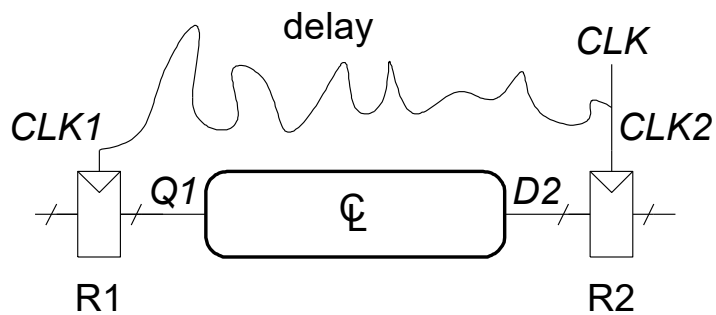
(30 + 50) ps > 70 ps ? **Yes!**

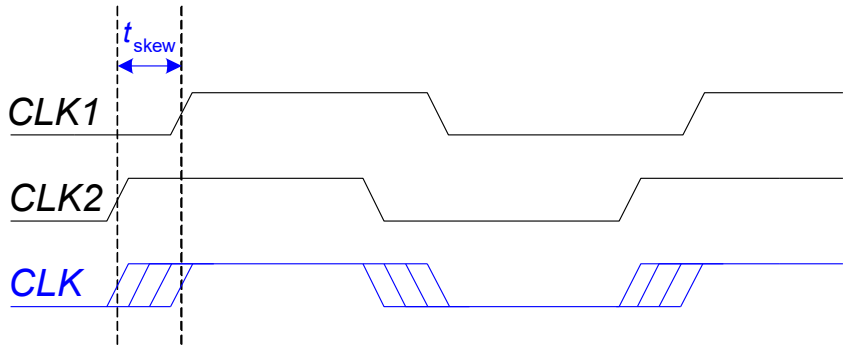
© Digital Design and Computer Architecture, 2nd Edition, 2012

ELSEVIER

Clock Skew

- The clock doesn't arrive at all registers at same time
- **Skew:** difference between two clock edges
- Perform **worst case analysis** to guarantee dynamic discipline is not violated for any register – many registers in a system!



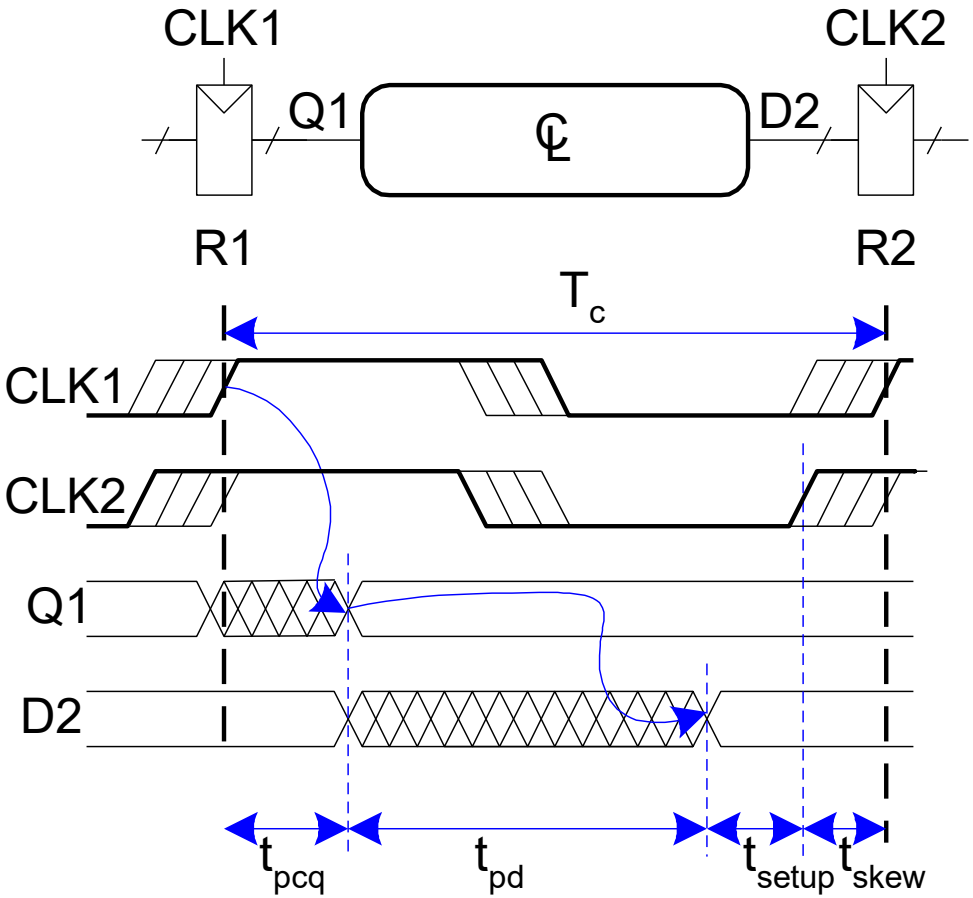


© Digital Design and Computer Architecture, 2nd Edition, 2012

ELSEVIER

Setup Time Constraint with Skew

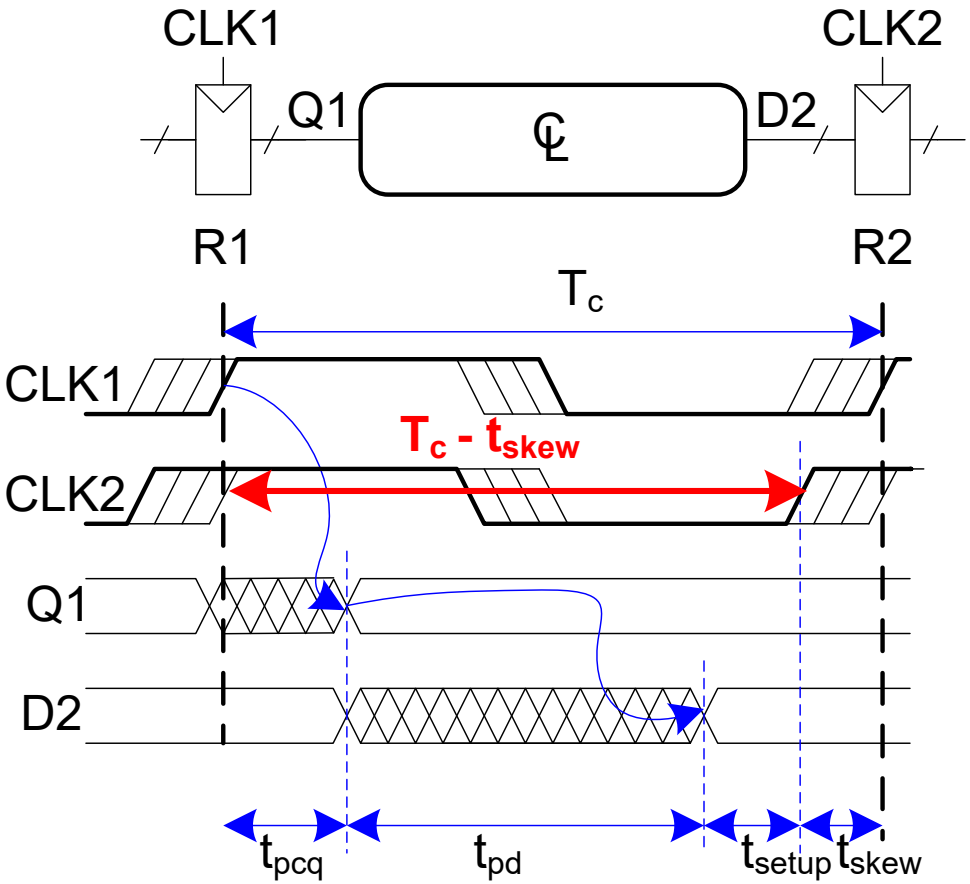
- In the worst case, CLK2 is earlier than CLK1



© *Digital Design and Computer Architecture*, 2nd Edition, 2012

Setup Time Constraint with Skew

- In the worst case, CLK2 is earlier than CLK1



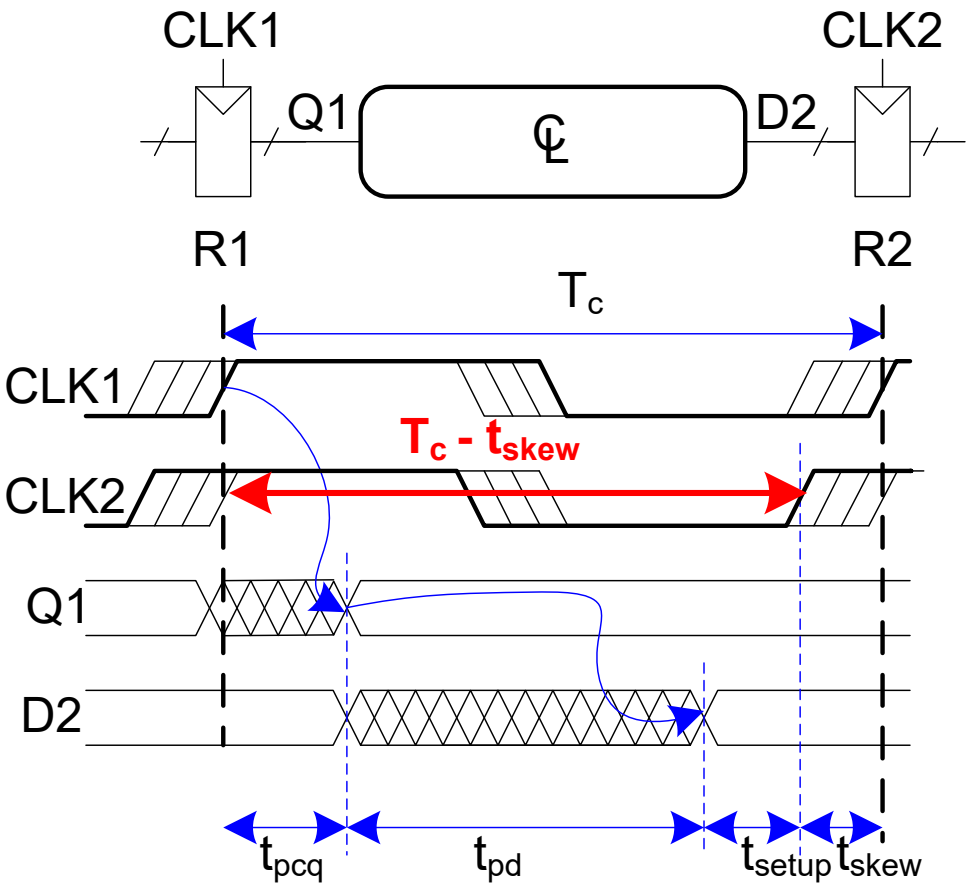
$$T_c - t_{skew} \geq t_{pcq} + t_{pd} + t_{setup}$$

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Setup Time Constraint with Skew

- In the worst case, CLK2 is earlier than CLK1



$$T_c - t_{skew} \geq t_{pcq} + t_{pd} + t_{setup}$$

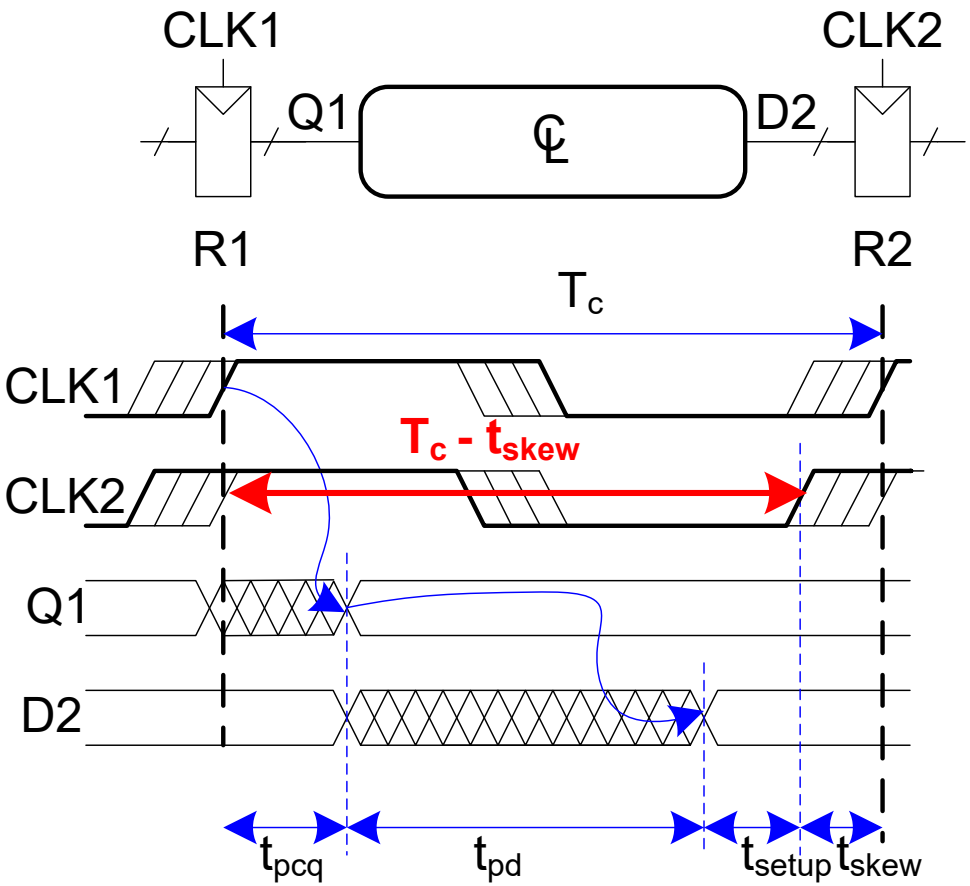
$$T_c \geq$$

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Setup Time Constraint with Skew

- In the worst case, CLK2 is earlier than CLK1



$$T_c - t_{skew} \geq t_{pcq} + t_{pd} + t_{setup}$$

$$T_c \geq t_{pcq} + t_{pd} + t_{setup} + t_{skew}$$

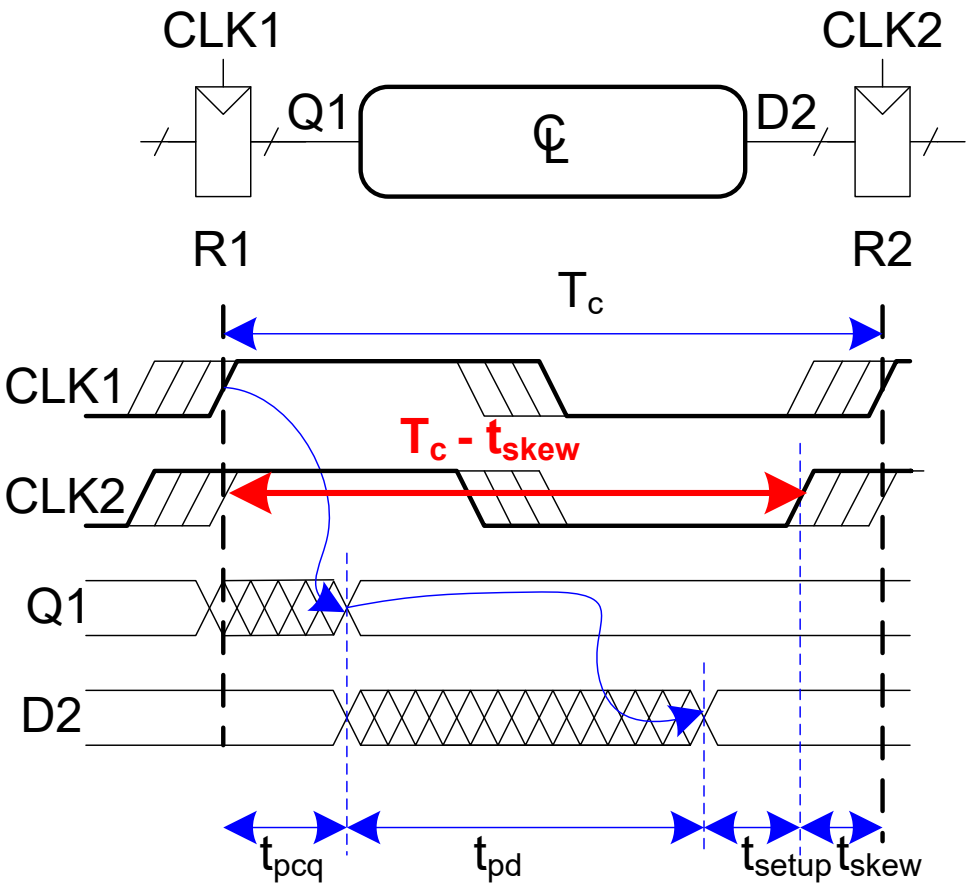
$$t_{pd} \leq$$

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Setup Time Constraint with Skew

- In the worst case, CLK2 is earlier than CLK1



$$T_c - t_{skew} \geq t_{pcq} + t_{pd} + t_{setup}$$

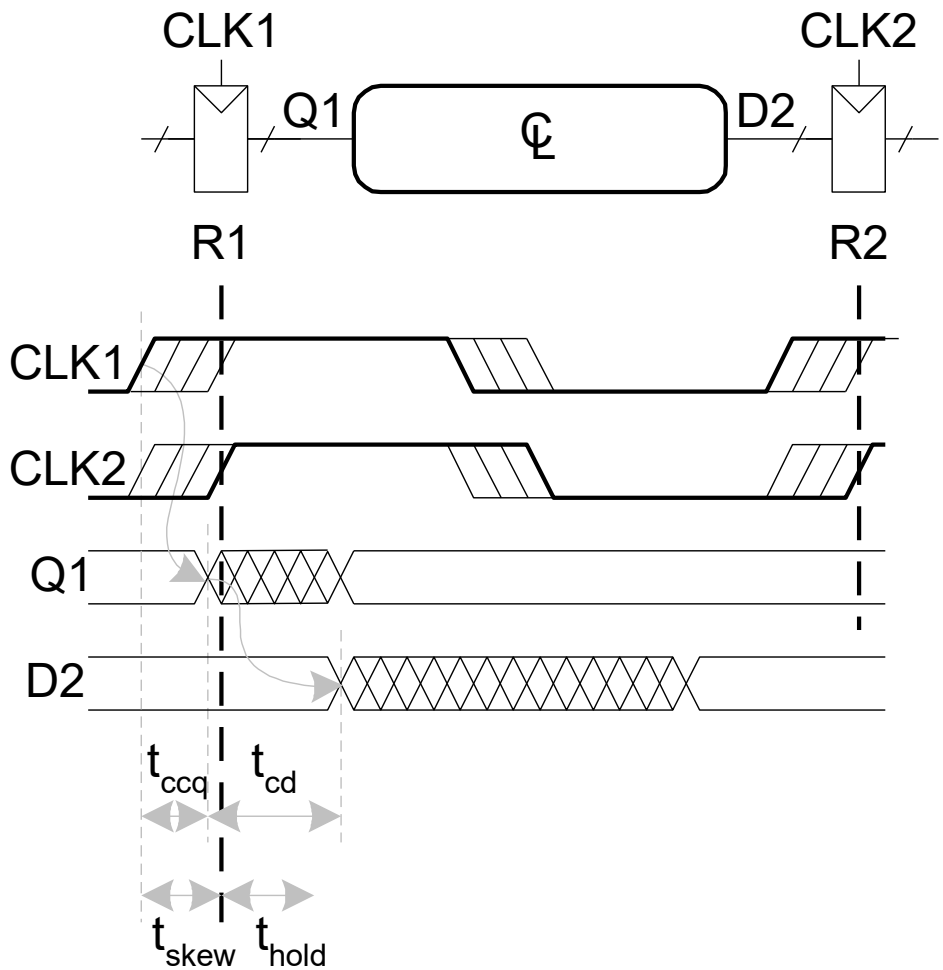
$$T_c \geq t_{pcq} + t_{pd} + t_{setup} + t_{skew}$$

$$t_{pd} \leq T_c - (t_{pcq} + t_{setup} + t_{skew})$$

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

Hold Time Constraint with Skew

- In the worst case, CLK2 is later than CLK1

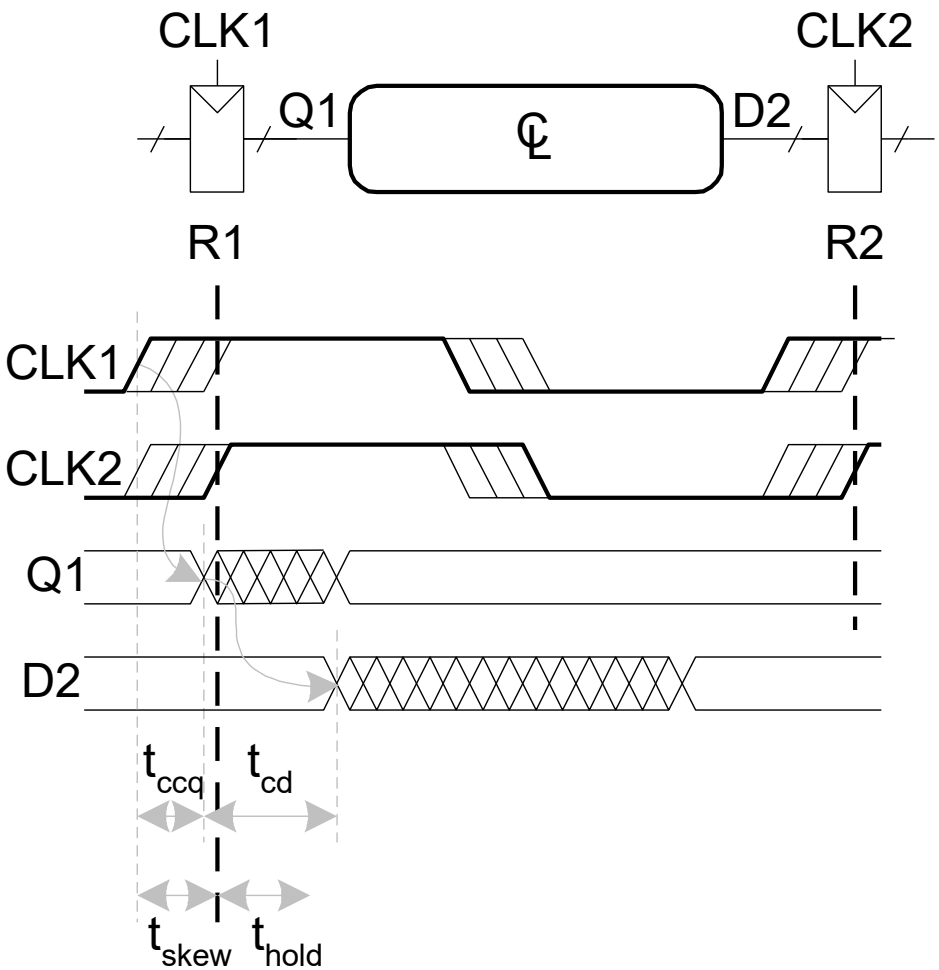


$$t_{ccq} + t_{cd} >$$

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

Hold Time Constraint with Skew

- In the worst case, CLK2 is later than CLK1



$$t_{ccq} + t_{cd} > t_{hold} + t_{skew}$$

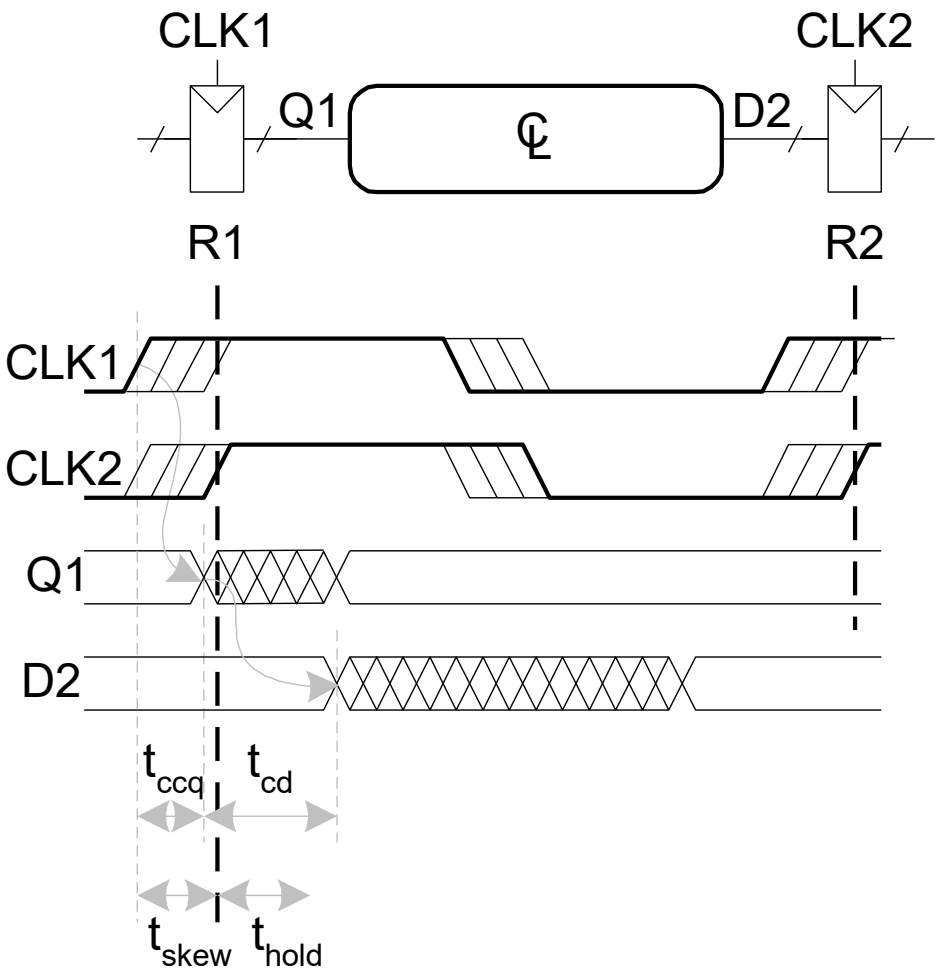
$$t_{cd} >$$

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Hold Time Constraint with Skew

- In the worst case, CLK2 is later than CLK1



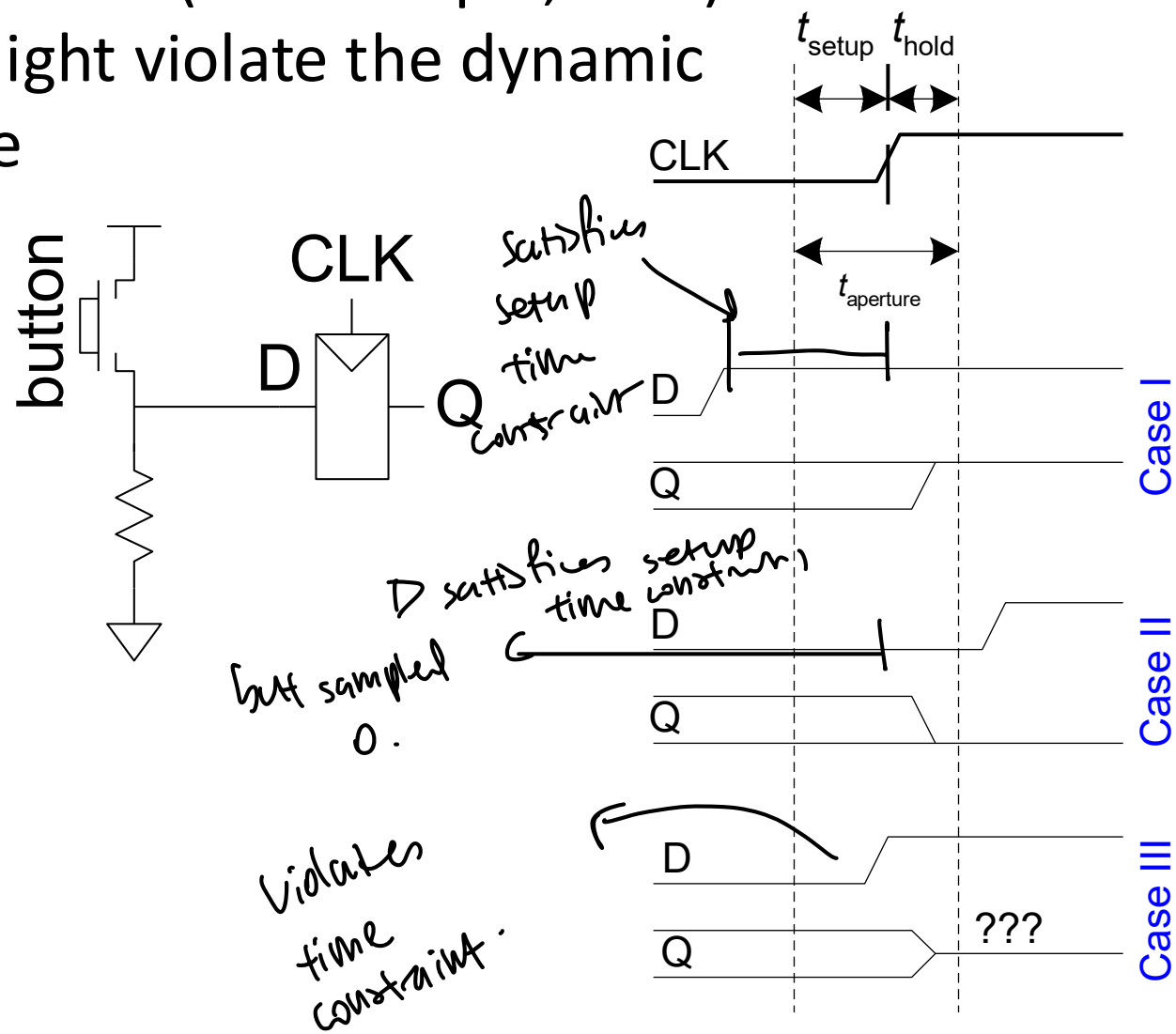
$$t_{ccq} + t_{cd} > t_{hold} + t_{skew}$$

$$t_{cd} > t_{hold} + t_{skew} - t_{ccq}$$

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

Violating the Dynamic Discipline

- Asynchronous (for example, user) inputs might violate the dynamic discipline



© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Sequential Logic

- Verilog uses certain coding styles to describe latches, flip-flops and FSMs
- Other coding styles may simulate correctly but produce incorrect hardware

Copyright © 2007 Elsevier

ELSEVIER

Always Statement

General Structure:

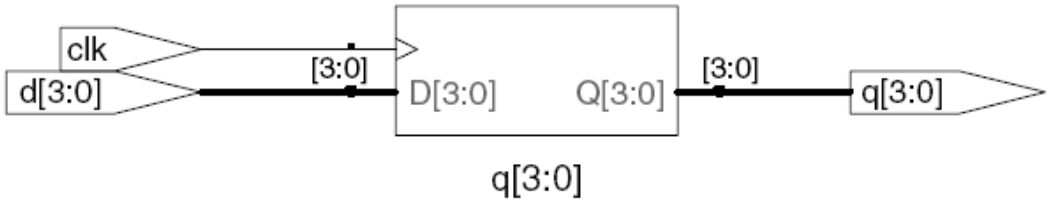
```
always @ (sensitivity list)
   statement;
```

Whenever the event in the `sensitivity list` occurs, the `statement` is executed

Copyright © 2007 Elsevier

D Flip-Flop

```
module flop(input              clk,
            input       [3:0]  d,
            output reg  [3:0]  q);

  always @ (posedge clk)
    q <= d;                    // pronounced "q gets d"

endmodule
```



Any signal assigned in an `always` statement must be declared `reg`. In this case `q` is declared as `reg`

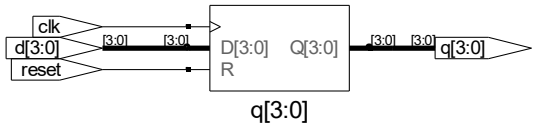
Beware: A variable declared `reg` is not necessarily a registered output. We will show examples of this later.

Copyright © 2007 Elsevier

Resettable D Flip-Flop

```
module flopr(input              clk,
             input              reset,
             input        [3:0] d,
             output reg   [3:0] q);

  // synchronous reset
  always @ (posedge clk)
    if (reset) q <= 4'b0;
    else       q <= d;

endmodule
```

flip-flop only sample @ the rising edge of the clock.

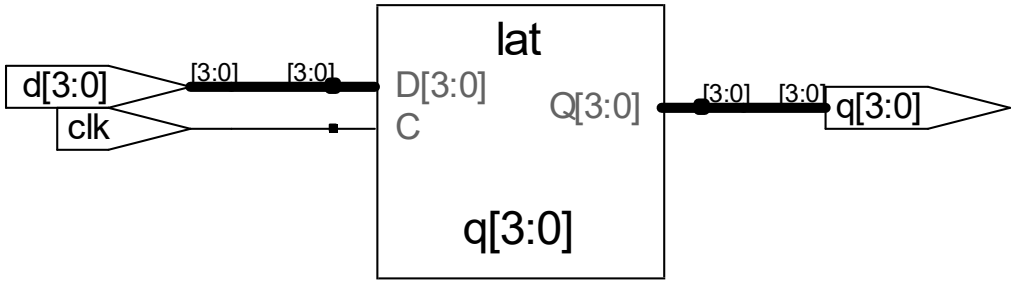


Copyright © 2007 Elsevier

Latch

```
module latch(input            clk,
             input      [3:0] d,
             output reg [3:0] q);

  always @ (clk, d)
    if (clk) q <= d;

endmodule
```

sample the input anytime clk = 1



Warning: We won't use latches in this course, but you might write code that inadvertently implies a latch. So if your synthesized hardware has latches in it, this indicates an error.

Copyright © 2007 Elsevier

Other Behavioral Statements

- Statements that must be inside `always` statements:
  - `if/else`
  - `case`, `casez`
- Reminder: Variables assigned in an `always` statement must be declared as `reg` (even if they're not actually registered!)

Copyright © 2007 Elsevier

ELSEVIER

Combinational Logic using **always**

```
// combinational logic using an always statement
module gates(input       [3:0] a, b,
             output reg [3:0] y1, y2, y3, y4, y5);
  always @(*)        // need begin/end because there is
    begin             // more than one statement in always
      y1 = a & b;     // AND
      y2 = a | b;     // OR
      y3 = a ^ b;     // XOR
      y4 = ~(a & b); // NAND
      y5 = ~(a | b); // NOR
    end
endmodule
```

This hardware could be described with assign statements using fewer lines of code, so it's better to use assign statements in this case.

Copyright © 2007 Elsevier

Combinational Logic using case

```
module sevenseg(input       [3:0] data,
                output reg  [6:0] segments);
  always @(*)
    case (data)
      //                      abc_defg
      0: segments = 7'b111_1110;
      1: segments = 7'b011_0000;
      2: segments = 7'b110_1101;
      3: segments = 7'b111_1001;
      4: segments = 7'b011_0011;
      5: segments = 7'b101_1011;
      6: segments = 7'b101_1111;
      7: segments = 7'b111_0000;
      8: segments = 7'b111_1111;
      9: segments = 7'b111_1011;
      default: segments = 7'b000_0000; // required
    endcase
endmodule
```

compare data with the following cases

Copyright © 2007 Elsevier

Combinational Logic using `case`

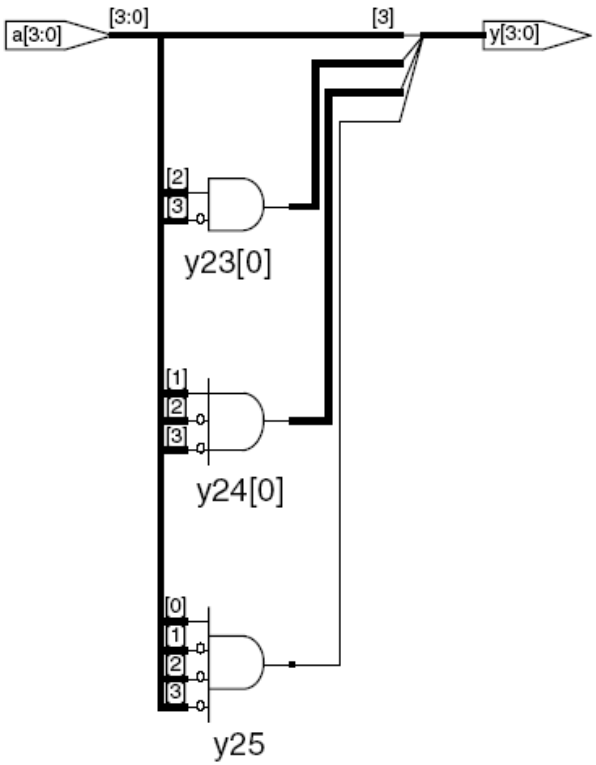
- In order for a `case` statement to imply combinational logic, all possible input combinations must be described by the HDL.
- Remember to use a **`default`** statement when necessary.

Copyright © 2007 Elsevier

ELSEVIER

Combinational Logic using casez

```
module priority_casez(input       [3:0] a,
                      output reg  [3:0] y);

  always @(*)
    casez(a)
      4'b1???: y = 4'b1000;   // ? = don't care
      4'b01??: y = 4'b0100;
      4'b001?: y = 4'b0010;
      4'b0001: y = 4'b0001;
      default: y = 4'b0000;
   endcase

endmodule
```



Copyright © 2007 Elsevier

Blocking vs. Nonblocking Assignments

- <= is a "nonblocking assignment"
  - Occurs simultaneously with others
- = is a "blocking assignment"
  - Occurs in the order it appears in the file

always use nonblocking.

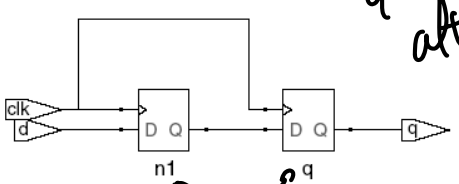
it's ok for combinational logic

don't write code like this.

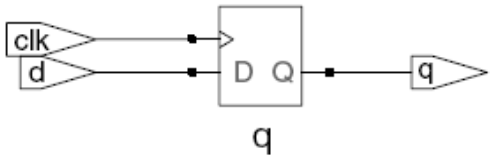
```
// Good synchronizer using
// nonblocking assignments
module syncgood(input      clk,
                input      d,
                output reg q);
  reg n1;
  always @(posedge clk)
    begin
      n1 <= d;  // nonblocking
      q  <= n1; // nonblocking
    end
endmodule
```

```
// Bad synchronizer using
// blocking assignments
module syncbad(input      clk,
               input      d,
               output reg q);
  reg n1;
  always @(posedge clk)
    begin
      n1 = d;  // blocking
      q  = n1; // blocking
    end
endmodule
```

q updated after 2 clock cycles



2 registers.



Copyright © 2007 Elsevier

Rules for Signal Assignment

- Use `always @(posedge clk)` and nonblocking assignments (<=) to model synchronous sequential logic

```
always @ (posedge clk)
  q <= d; // nonblocking
```

- Use continuous assignments (assign …)to model simple combinational logic.

```
assign y = a & b;
```

- Use `always @ (*)` and blocking assignments (=) to model more complicated combinational logic where the `always` statement is helpful.

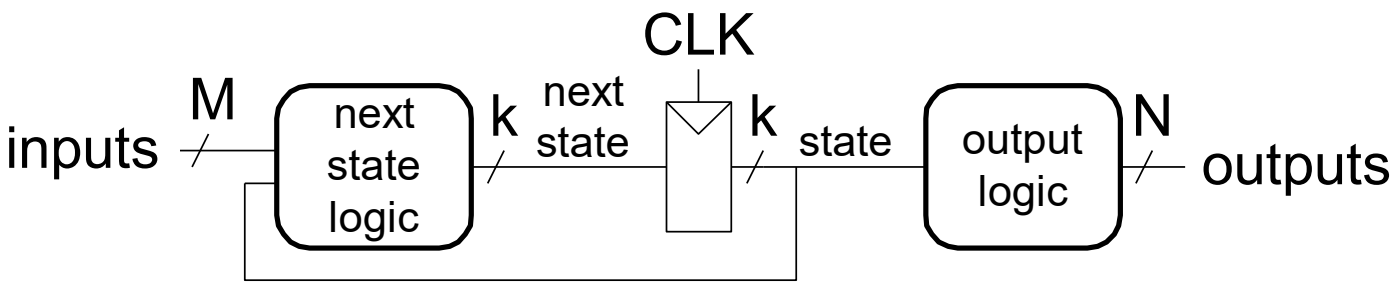
- Do not make assignments to the same signal in more than one `always` statement or continuous assignment statement.

Copyright © 2007 Elsevier

ELSEVIER

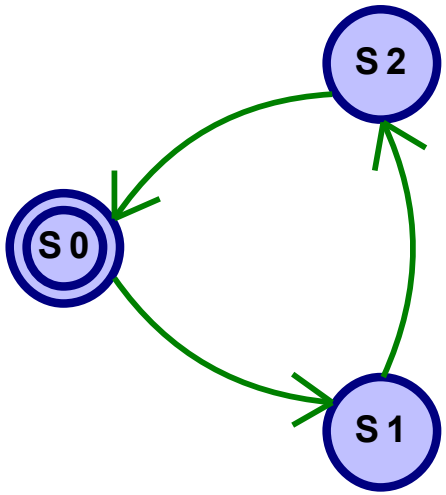
Finite State Machines (FSMs)

- Three blocks:
  - next state logic
  - state register
  - output logic



Copyright © 2007 Elsevier

FSM Example: Divide by 3



The double circle indicates the reset state

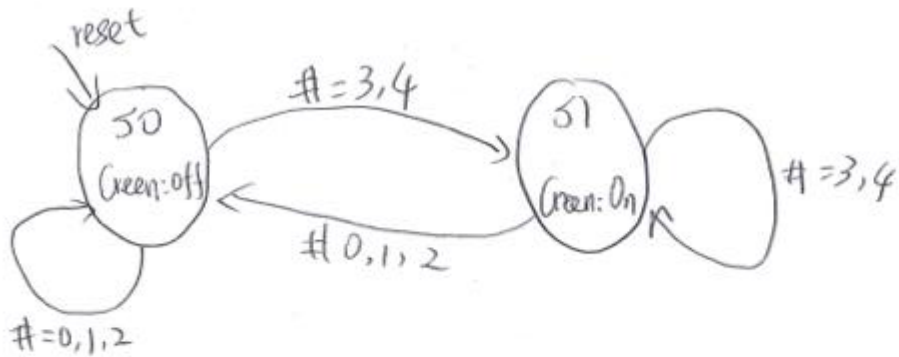
Copyright © 2007 Elsevier

FSM in Verilog

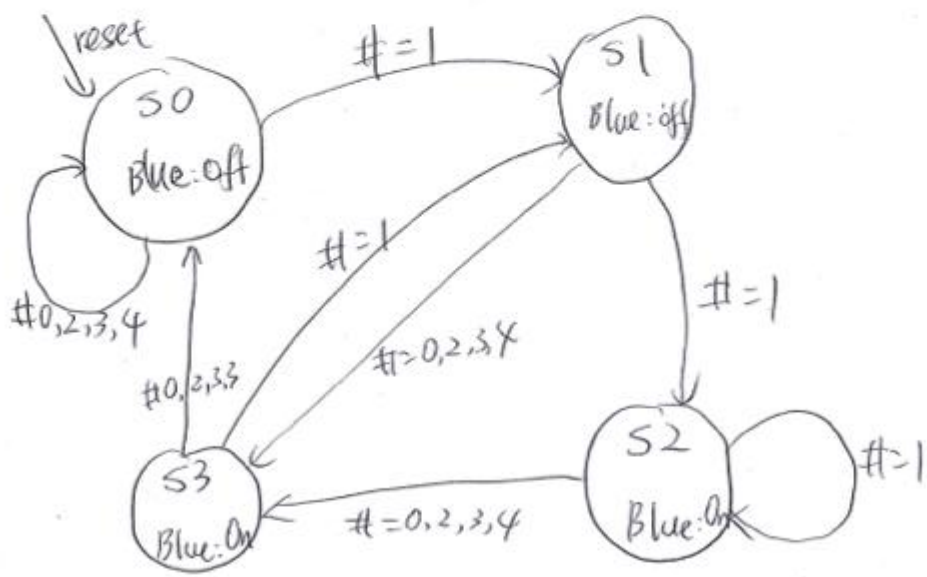
```
module divideby3FSM (input  clk,
                     input  reset,
                     output q);
   reg  [1:0] state, nextstate;

   parameter S0 = 2'b00;
   parameter S1 = 2'b01;
   parameter S2 = 2'b10;

// state register
   always @ (posedge clk, posedge reset)
      if (reset) state <= S0;
      else       state <= nextstate;
 // next state logic
   always @ (*)
      case (state)
         S0:      nextstate = S1;
         S1:      nextstate = S2;
         S2:      nextstate = S0;
         default: nextstate = S0;
      endcase
   // output logic
   assign q = (state == S0);
endmodule
```

Copyright © 2007 Elsevier

Green Light FSM



Blue Light FSM



Green FSM

current state	input NYBS	next state
S0 /00	0,1,2	S0 /0
S0 /00	3,4	S1 /1
S1 /1	0,1,2	S0 /0
S1 /1	3,4	S1 /1

~~0,1,2 = $\bar{N}\bar{Y}\bar{B}\bar{S}$ +~~ C 3,4 = ~~$\bar{Y}\bar{B}\bar{S}$ +~~ $N\bar{Y}B + \bar{Y}BS + N\bar{Y}S + NBS$

$\bar{C}$ = 0,1,2 = $\overline{3,4}$

Blue

current state	input	next state
S0 /00	0,2,3,4	S0 /00
S0 /00	1	S1 /01
S1 /01	0,2,3,4	S3 /11
S1 /01	1	S2 /10
S2 /10	0,2,3,4	S3 /11
S2 /10	1	S2 /10
S3 /11	0,2,3,4	S0 /00
S3 /11	1	S1 /01

C, #1 = $N\bar{Y}\bar{B}\bar{S} + \bar{N}\bar{Y}\bar{B}\bar{S} + \bar{N}\bar{Y}B\bar{S} + \bar{N}\bar{Y}\bar{B}S$

$\bar{C}$, #0,2,3,4

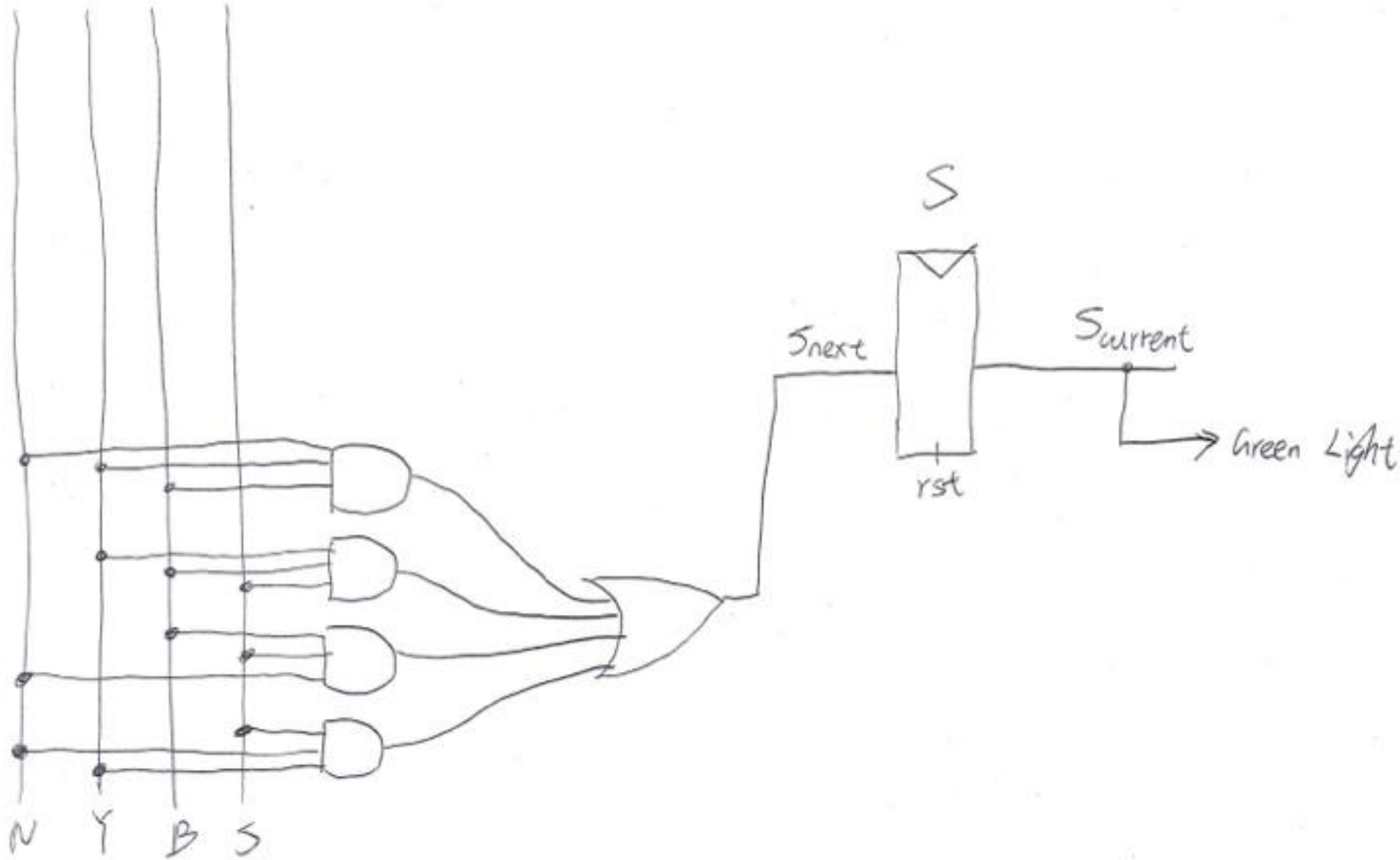
Green FSM / output table

Current State	Output
S0 /0	Green : off
S1 /1	Green : On

Blue FSM

current state	output
S0	Blue : off
S1	Blue : off
S2	Blue : On
S3	Blue : On

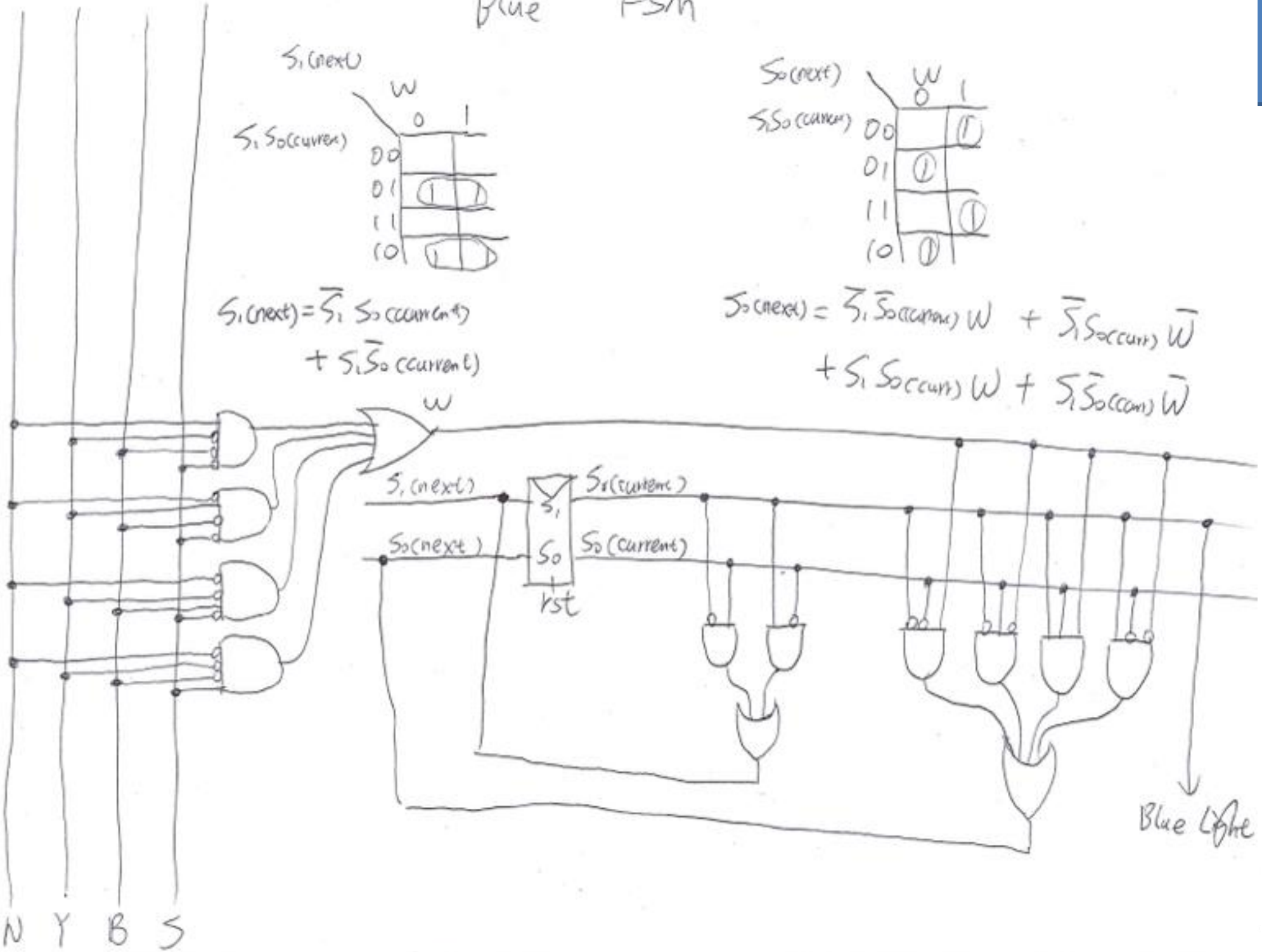
Green FSM



Blue FSM

$$S_1(next) = \bar{S}_1 S_0(current) + S_1 \bar{S}_0(current)$$

$$S_0(next) = \bar{S}_1 \bar{S}_0(current) W + \bar{S}_1 S_0(curr) \bar{W} + S_1 S_0(curr) W + S_1 \bar{S}_0(curr) \bar{W}$$



Testbenches

- HDL code written to test another HDL module, the *device under test* (dut), also called the *unit under test* (uut)
- Not synthesizeable
- Types of testbenches:
  - Simple testbench
  - Self-checking testbench
  - Self-checking testbench with testvectors

Copyright © 2007 Elsevier

ELSEVIER

Simple Testbench

```
module testbench1();
  reg  a, b, c;
  wire y;
  // instantiate device under test
  sillyfunction dut(a, b, c, y);
  // apply inputs one at a time
  initial begin
    a = 0; b = 0; c = 0; #10;
    c = 1; #10;
    b = 1; c = 0; #10;
    c = 1; #10;
    a = 1; b = 0; c = 0; #10;
    c = 1; #10;
    b = 1; c = 0; #10;
    c = 1; #10;
  end
endmodule
```

Copyright © 2007 Elsevier

Self-checking Testbench

```
module testbench2();
  reg  a, b, c;
  wire y;
  // instantiate device under test
  sillyfunction dut(a, b, c, y);
  // apply inputs one at a time
  // checking results
  initial begin
    a = 0; b = 0; c = 0; #10;
    if (y !== 1) $display("000 failed.");
    c = 1; #10;
    if (y !== 0) $display("001 failed.");
    b = 1; c = 0; #10;
    if (y !== 0) $display("010 failed.");
    c = 1; #10;
    if (y !== 0) $display("011 failed.");
    a = 1; b = 0; c = 0; #10;
    if (y !== 1) $display("100 failed.");
    c = 1; #10;
    if (y !== 1) $display("101 failed.");
    b = 1; c = 0; #10;
    if (y !== 0) $display("110 failed.");
    c = 1; #10;
    if (y !== 0) $display("111 failed.");
  end
endmodule
```

Copyright © 2007 Elsevier

Testbenches

- HDL code written to test another HDL module, the *device under test* (dut), also called the *unit under test* (uut)
- Not synthesizeable
- Types of testbenches:
  - Simple testbench
  - Self-checking testbench
  - Self-checking testbench with testvectors
- **Read textbook to learn how to write testbenches!**

Copyright © 2007 Elsevier

ELSEVIER